Opgave 4g

Christer K. Lindholm, Carl-Emil Rohde og Emil Olesen

October 2022

4g0.

Opsummering af problemformuleringer

Løsningsmuligheder, overvejelser og argumenter

4g0(a): Distance mellem source og target

I den første delopgave, skal vi skrive en funktionen "dist: p1: pos -> p2: pos -> int", der givet to koordinatsæt - et startpunkt "source"og slutpunkt "target", udregner distancen mellem de to punkter. Til at gøre dette brugte vi formlen $distance = (x2 - x1)^2 - (y2 - y1)^2$. Vores funktion "dist"tager to argumenter i form af tupler med koordinatsæt, hvor første og andenkoordnaterne identificeres ved brug af "fst"og "snd"kommandoen, bliver sat ind i formlen. For at funktionen vil udregne distancen, bliver vi nødt til at omregne fra heltal til kommatal. Dette gør vi ved, at "støbe"regnestykkerne i parantes med kommandoen "float-nøgleordet. Funktionen fra opgaven påskriver dog, at funktionen returnerer heltal. Derfor som det sidste før vi printer resultatet, støber vi typen tilbage til heltal med "int()".

4g0(b): Tættere på target, med candidate

Vi skal i denne opgave skrive et program med typerne *candidates: src: pos -> tg: pos -> pos list* til at bringe robotten tættere på target. I denne programmeringsdel tager vi fortsat udgangpunkt i *source* og *target* som værende henholdvis $(3, 3)$ og $(1, 1)$. Vores tankegang er, at lægge 1 til og trække 1 fra koordinaterne på *source* for at få vores næste *candidate*. Efterfølgende vil vi bruge *List.filter* med en anonym funktion til, at vise den/de lister, der har mindre eller samme afstand til target, som afstanden mellem source og target. Resultatet ser ud som følgende.

```
Candidates = [(2, 3); (3, 2)]
```

Figur 1:

Vi kan her se, at vi får to mulige udfald, der begge vil bringe os tættere på *target*.

4g0(c): Samtlige korteste ruter til target

Vi skal i denne opgave bygge et program, der fortæller brugeren samtlige muligheder for at komme fra *source* til *target* på den kortest mulige måde. Vi skal anvende programmeringsstrukturen: *routes: src: pos -> tg: pos -> pos list list* - hvor en funktion, *routes* der som input tager to positioner der henholdsvis er vores *source*

og *target*, for at returnere positioner som lister i en liste. Det vil sige at, *routes* skal returnere step by step, alle de ruter man kan benytte, for hurtigst at komme til *target*.

Vi har lavet en rekursiv funktion, der bruger *match with* til returnere en liste i en liste, når kordinaterne i listen = *target*. For alle andre udfald skal den bruge List.concat samt List.map, til at gennemgå alle andre muligheder, samt sortere alle ruter fra, der er længere end de korteste. Eftersom der findes flere måder hvorpå man kan bevæge sig fra a til b, og stadig have tilbagelagt samme afstand, vil programmet returnere flere anvendelige ruter, der stadig opfylde kriteriet for hurtigste rute. Man kunne forestille sig, at havde afstanden mellem *source* og *target* været tilpas stor, ville mulighederne tinærme sig et voldsomt stort tal, eftersom kombinationerne ville være mange.

```
Routes = [[(3, 3); (2, 3); (1, 3); (1, 2); (1, 1)];
 [(3, 3); (2, 3); (2, 2); (1, 2); (1, 1)];
 [(3, 3); (2, 3); (2, 2); (2, 1); (1, 1)];
 [(3, 3); (3, 2); (2, 2); (1, 2); (1, 1)];
 [(3, 3); (3, 2); (2, 2); (2, 1); (1, 1)];
 [(3, 3); (3, 2); (3, 1); (2, 1); (1, 1)]]
```

Figur 2:

Vi kan her se de korteste ruter fra *source* til *target*, angivet som tuples i lister, i en liste.

4g0(d): Korteste ruter med diagonale veje

I denne opgave skal funktionen *candidates* gøres i stand til også at bruge diagonale koordinater som bevægelser mod målet, mens *routes* kun skal returnere de korteste ruter.

Candidates var allerede bygget til at gennemgå lister af mulige bevægelser samt til at filtrere dem fra, som ikke går tættere på target. For at indføre diagonale muligheder tilføjede vi blot nye lister til rækken af lister, den vurderer. Diagonale bevægelser svarer til lægge 1 til eller trække en fra i både første- og andenkoordinat samtidigt.

Routes begynder nu at returnere et dramatisk øget antal mulige stier mod target. For at begrænse outputtet til de korteste, brugte vi listemodulets indbyggede funktioner:

1. Vi binder den sammensatte liste af alle de mulige ruter fra *routes* til navnet *cons*.
2. List.map funktionen løber igennem *cons* og bruger List.length på hvert element. Den danner dermed en liste med antallet af bevægelser (tupler) i hver sti til målet.
3. På listen med antallet af bevægelser, finder vi de korteste med List.min.
4. Endelig kan List.filter løbe den originale *routes*-liste igennem, mens den læser hver listes længde med List.length samt sortere alle lister fra, der ikke lig længden på den korteste stis liste.

```
let rec routes (src: pos) (tg: pos) : pos list list =
match src with
    p when p = tg -> [[tg]]
    | _ ->
        let allRoutes = List.map (fun elm ->
                        List.map (fun elm2 -> src::elm2) (routes elm tg)) (candidates src tg)
        let cons = List.concat allRoutes
        let mapList = List.map (fun list -> List.length list) (cons)
        let minimum = List.min mapList
        let filter = List.filter (fun elm -> List.length elm = minimum) (cons)
        filter
```